BIL 322 · FINAL PROJECT · PHASE 1 LAB

Autonomous Drone Swarm Choreographers

Threads, queues, locks, barriers, and one captain whose battery dies.



NOTES

These slides are the briefing for the Phase 1 lab of the final project. The notes under each slide are for you: read them after the lab, together with the Phase 1 handout and the project description. The final project has three phases of about one week each. Phase 1 is the thread and synchronization part. The colored dots are the drones; at the end of the project they form a picture in the sky.

THE PROJECT

A swarm of drones that dances to music

Hundreds of LED drones perform a choreography, then form a picture in the sky. Four roles make the show.

Center

Artistic Director

Reads the score and the target image, sets each section’s theme, runs the beat clock, hosts the slow planner.

Leaders

Dance Captains

One process per zone. Each asks for a plan, checks it, and directs its troupe.

Followers

Performers

Threads inside a leader in Phase 1. Each flies to a cell, lights a color, reports.

Viewer

Broadcast Booth

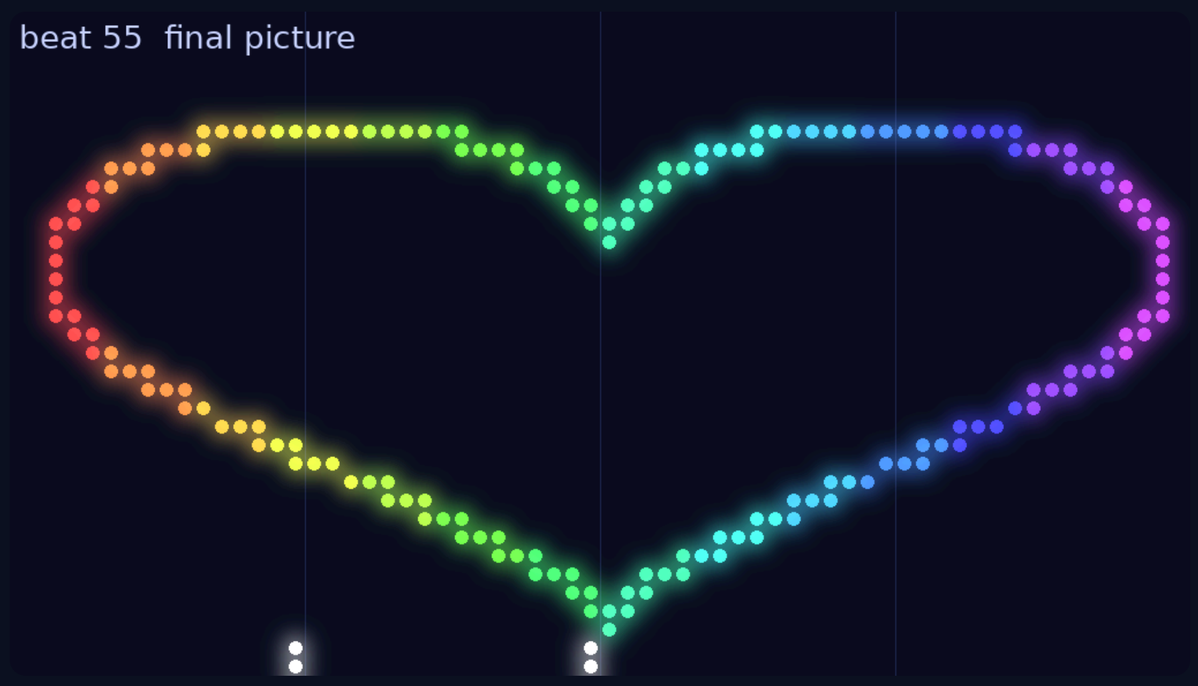
A separate GUI from Phase 2. It may drop frames, never slow the swarm.

NOTES

The story: the engineering faculty takes a stadium tradition into the air. The center knows the score (the sections and moods of the music) and the target image, and decides only the coarse theme of each section. The detailed movements come from the choreographer, a slow thinker. In a real show it would be an LLM; here it is a stub that sleeps, so you feel what it is like to wait for a model that is slow, busy or wrong. Leaders are dance captains, one process per zone. Followers are cheap performers; in Phase 1 they are threads inside their captain. The viewer becomes a separate GUI in Phase 2. Today it is just the stage drawn as text.

THE GOAL

What it looks like when it works



4 captains × 46 performers = **184 drones**. Every dot is a drone, every pixel of the heart gets one.

The 4 white dots at the bottom are spare drones that stay home.

```
./center --score data/scores/show.score --
followers 46 --ascii 8
```

Phase 2 draws this in a real window.

NOTES

This is the end goal of Phase 1, drawn from the center’s text output. The animation plays the whole show: all drones start in their home rows at the bottom (each zone is one captain’s troupe), then a wave formation, then the finale where the drones fly to the pixels of the target image, a rainbow heart. The numbers: 4 leaders times 46 followers is 184 drones for 180 pixels, so four spare drones stay at home. The command reproduces it once your tasks work. Until then the show does not run end to end. Phase 2 draws the same thing in a real window (SDL, GTK or Qt).

THREE PHASES, ONE SYSTEM

Same protocol, three transports

	Phase 1 (this week)	Phase 2	Phase 3
Leader	child process (fork, exec)	child process	client process
Performer	thread in its captain	thread in its captain	separate process
Links	pipes	pipes + shared memory	TCP sockets
Viewer	text in the terminal	own process, shared memory	socket client
Captain dies	center notices and restarts it	+ its cells are released	orphans are rehomed

Write your send and receive code against **file descriptors**, and keep parsing separate from transport: Phase 3 then changes the transport, not the program.

NOTES

The protocol stays the same in all phases; what changes is how the pieces talk. This week leaders are child processes created with fork and exec (your Lab 4 pattern) and connected by pipes (Lab 5). Performers are threads. Phase 2 moves the stage into shared memory and the viewer into its own process. In Phase 3 everything uses sockets and performers become separate processes, so when a captain dies its troupe is really orphaned. The question behind the whole project: when a captain dies, who notices, and who takes over its troupe? Your code already sends lines over file descriptors, so the later change of transport is small.

PHASE 1 ARCHITECTURE

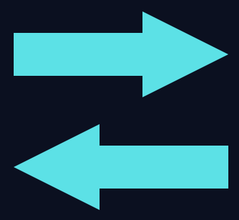
Two kinds of process, many threads

center

one process, the parent

- **main thread** · clock loop: THEME, BEAT, waits for READY and ACK, audits the stage
- **reader threads** · one per leader, parse its pipe
- **planner pool** · threads behind a bounded request queue, Lab 6 pool + arena

pipes



one text line per message

leader × 4

one child process per zone: fork, dup2, exec

- **main thread** · reads the center’s lines
- **thinker** · PLAN_REQ, validate, READY
- **follower threads** · F0 … Fk, one barrier per beat, claim cells in the zone’s occupancy grid

BIL 322 · Swarm Phase 1 lab

NOTES

The center’s main thread runs the clock: it sends each section’s theme and each beat, and waits until every leader has said READY for a section and ACK for each beat. One reader thread per leader parses that leader’s pipe. The choreographer is a thread pool behind a bounded request queue, and every request takes memory from your Lab 6 pool. Each leader is a child process started with fork, dup2 and exec, like your Lab 4 shell. Inside it: a main thread reading the pipe, a thinker thread that asks for plans, and one thread per follower. The occupancy grid is per leader, because the followers are its threads and can claim cells directly. The center does not trust the leaders: it audits the POS reports for collisions and speed. Read center.c once from top to bottom before you write your part; its locks and their order are commented.

THE PROTOCOL (FROZEN)

One message per line, plain text

center → leader

`ASSIGN id x0 y0 x1 y1 n`	your zone
`THEME sec start end theme mood`	coarse direction
`PLAN req n`	then n keyframe lines
`BUSY req retry_ms`	queue full, later
`BEAT n`	start beat n
`SHUTDOWN`	bye

leader → center

`HELLO id n`	I am here
`PLAN_REQ req sec n seed`	give me a plan
`READY id sec`	plan is loaded
`POS id beat f x y c`	where f is
`ACK id beat`	beat done

- Keyframe line: **`beat follower x y color`**. A plan has one for every beat and every follower.
- Unknown message: log and ignore. Malformed: reject, never crash. Lines are at most 256 bytes.
- You can test a leader by typing these lines into its stdin.

NOTES

The protocol is a fixed contract. Every message is one plain-text line, so humans and agents can both read it: you can watch a rehearsal scroll by, or type a message into a leader’s stdin by hand to test it. From the center: ASSIGN gives a leader its zone, THEME gives the coarse direction of a section, PLAN carries the detailed choreography as n keyframe lines, BUSY says the request queue is full, BEAT is the clock, SHUTDOWN ends the show. From the leader: HELLO, PLAN_REQ to commission a plan, READY when the plan is loaded, POS and ACK to report where each follower is after a beat. A plan is dense: one keyframe for every beat and every follower. Unknown messages are logged and ignored; malformed ones are rejected and never crash the receiver. The parser is given; you use it. See Section 3 of the project description.

HOW IT WORKS

One beat, step by step

1. Center sends **BEAT b** (only after the ACK for b−1).
2. Each follower finds its target for b in the stored plan (no plan: hold).
3. **Release** the cell you are in, then wait at the **barrier**.
4. **Claim** the target cell in the grid, then the **barrier** again.
5. The thread the barrier names sends the **POS** lines and **ACK b**.
6. Center audits collisions and speed, then sends BEAT b+1.

Why two barriers?

In one beat, follower A may move into the cell follower B is leaving.

All releases must happen before any claim, or A's claim fails by accident.

A failed claim is a safety violation, never something to retry in a loop.

NOTES

This is the heart of Tasks 3 and 5. The center sends BEAT b to every leader, and never sends b+1 before it has the ACK for b: that keeps everyone in lockstep. In the leader, the main thread publishes the beat; each follower wakes up, looks up its target in the stored plan, and holds its position if there is no plan. Then two phases separated by barriers. Phase 1: everyone releases the cell it is in; barrier. Phase 2: everyone claims its target cell; barrier. Why two? Followers move at the same time. If A wants the cell that B is leaving, A's claim would fail if B had not released it yet. With a barrier in between, all releases happen before any claim. After the second barrier, pthread_barrier_wait returns PTHREAD_BARRIER_SERIAL_THREAD to exactly one thread; that thread sends the POS lines for all followers and then the ACK. A failed claim is a safety violation, never something to retry in a loop. Think about it: what happens if one follower returns early and skips a barrier? Everyone else waits forever.

WHAT YOU HAVE TO DO

Six tasks in the skeleton

#	Task	Where	Points
1	Bounded queue	`queue.c`	20%
2	Choreographer and captain	`center.c, leader.c (thinker)`	25%
3	Performers and the grid	`grid.c, leader.c (follower)`	25%
4	Find the deadlock	`grid_swap, swapstress`	10%
5	Captain down	`center.c (on_leader_exit)`	10%
6	Tests and report	`make test, make tsan, REPORT.md`	10%

Given: protocol parser, planner, validator, loaders, the clock loop, the unit tests and the test script. **Yours:** the TODO blocks.

NOTES

This is the whole assignment on one page. The skeleton repository already contains the protocol parser, the planner (the slow stand-in for an LLM), the plan validator, the score and image loaders, the center’s clock loop and the tests. You fill in the TODO blocks. Task 1: the bounded queue. Task 2: the request pipeline in the center and the thinker in the leader. Task 3: the occupancy grid in both lock modes and the follower threads. Task 4: a deadlock that is already in grid_swap; you reproduce it, explain it and fix it. Task 5: notice a dead captain and restart it. Task 6: make test and make tsan must pass, plus a short report. The handout has the details of each task.

TASK 1 · 20%

The bounded queue

```
queue_t *queue_create(size_t cap, size_t item_size);
int queue_push(queue_t *q, const void *item); // blocks if full
int queue_try_push(queue_t *q, const void *item); // -1 if full
int queue_pop(queue_t *q, void *out); // blocks if empty
void queue_close(queue_t *q); // wake everyone
size_t queue_size(queue_t *q); void queue_destroy(queue_t *q);
```

Used twice: the center's **request queue** (try_push, then BUSY if full) and the leader's queue of themes.

Rules

- **while**, not if, around every pthread_cond_wait
- After close: pop drains the rest, then returns −1; push returns −1
- close wakes **everyone**: broadcast
- No busy-waiting, no sleep

NOTES

Task 1 is the classic bounded buffer from the lecture: a ring buffer with a mutex and condition variables. Two condition variables, not full and not empty, make the logic easy to read. Items are copied by value, so the same queue holds ints, requests and themes. Use while because of spurious wakeups and because another thread may take the item before you. After queue_close, pop keeps returning the remaining items and then fails, so workers drain the queue and exit; push fails immediately, also for a thread that was already blocked in push. close must wake everybody: broadcast, not signal. No busy-waiting and no sleep. You can test the queue alone with tests/test_queue before the rest of the project exists.

TASK 2 · 25%

The choreographer and the captain

Center

handle_plan_req, serve_request

- try_push; queue full: **BUSY 50**. A reader thread never blocks
- **Semaphore**: at most --max-inflight planner calls
- KV blocks from the Lab 6 pool. The pool is **not thread-safe**: wrap it in pool_mu
- Pool empty: BUSY. That is backpressure too
- Every exit path gives back the blocks and posts the semaphore

Captain

thinker_main

- Per THEME: send **PLAN_REQ**, wait for PLAN or BUSY
- BUSY: sleep retry_ms + a **random** extra. A fixed delay makes every leader retry together: livelock
- Run **validate_plan**. Invalid: new seed, at most 6 tries (--flaky 50 tests this)
- Always send **READY**, even after giving up: the troupe holds and the show goes on

BIL 322 · Swarm Phase 1 lab

NOTES

In the center, a reader thread receives PLAN_REQ and must never block: it does a try_push on the request queue, and if the queue is full it answers BUSY immediately. A planner thread pops the request, takes a semaphore slot (this limits how many planner calls run at once) and takes its KV blocks from your Lab 6 pool. Your pool is deliberately not thread-safe, so every alloc and free goes under pool_mu. If the pool is empty the answer is BUSY as well: two kinds of backpressure. Then plan_section runs (it sleeps to simulate a slow model), the PLAN is sent, and the blocks and the semaphore are released on every path. In the leader, the thinker asks for a plan per theme. After BUSY it waits retry_ms plus a random extra: if all leaders retried at the same moment they would collide again and again, which is livelock. A real model could be wrong, so the captain never trusts a plan: the validator checks speed, collisions, zone and density, and an invalid plan is requested again with a new seed. The flaky option makes half of the plans wrong on purpose. READY is sent even if the captain gave up; otherwise the center would wait for that leader forever.

TASK 3 · 25%

Performers in lockstep, and the grid

The occupancy grid

grid_claim, grid_release, grid_owner

- One drone per cell, per leader’s zone
- Two modes: **one mutex** for the grid, or **one mutex per row**
- A reader must lock too. Why?
- make bench times both with 1, 2, 4, 8 threads. Put the table in the report and **explain** it

follower_main

for every beat b, in order

```
wait for BEAT b
target = plan or hold
release(old cell)
barrier
claim(target)
barrier
serial thread: POS lines, ACK
```

Failed claim = safety violation: count it, log it, stay where you were.

NOTES

The occupancy grid: one mutex for the whole grid versus one mutex per row. Implement claim, release and owner for both modes. A reader must lock too: without the lock it can see a half-updated state, which is a data race. make bench runs random claim and release from 1, 2, 4 and 8 threads in both modes. Put the table in your report and explain your own numbers: they depend on your machine and cores (contention on one mutex, what scales, and what else could be the bottleneck). The show itself at tempo 0 is dominated by the planner’s thinking time, so the grid is not the bottleneck there. Second part: follower_main with the two barriers from the beat slide. With a valid plan a claim never fails; if it does, it is a safety violation, so count it and log it.

TASKS 4 AND 5 · 10% EACH

A deadlock to find, a captain to lose

4 · Find the deadlock

```
./swapstress row 8 2
```

- grid_swap locks two rows. In row mode it **hangs**
- Show the stuck threads: gdb -p, thread apply all bt
- Name the Coffman condition you break; fix it with a lock ordering
- Explain why your fix cannot livelock

5 · Captain down

```
./center --battery 1:20
```

- Notice by **EOF on the pipe**, not by polling. Print “captain 1 went dark”
- **waitpid**, close the pipes: no zombie
- Wake the clock loop (st_cv) so it stops waiting
- Restart: ASSIGN, THEME from the next beat; the troupe starts at home

BIL 322 · Swarm Phase 1 lab

NOTES

Task 4: the given grid_swap exchanges two cells and locks both rows. If two threads swap the same pair of rows in opposite orders, each holds one lock and waits for the other: a deadlock. swapstress reproduces it in seconds in row mode; in global mode it does not hang. Show the stuck threads in gdb, name the Coffman condition you break (circular wait, broken by a lock ordering: always lock the lower row first), and explain why your fix cannot cause a livelock. Task 5: the leader started with --battery 1:20 exits when beat 20 starts, as if it crashed. The center must notice through EOF on the pipe, reap the process with waitpid so there is no zombie, close both pipe ends, and wake the clock loop that may be waiting for an ACK from it. With restart (the default) it starts a new captain for the zone: ASSIGN, then THEME from the next beat on. The new troupe starts at home, so the speed audit forgives one jump (skip_speed), but collisions are never forgiven. With --no-restart the zone is just marked gone. In the report: what would be different if the performers were processes? That is Phase 3.

TASK 5, AS IT SHOULD LOOK

Captain 1 dies, the show goes on



Beat 24: the second troupe is back at home (white): a new captain, new threads.

Beat 25: the new captain’s plan starts, and the troupe dances with the others.

Beat 55: the picture still forms, with zero collisions and zero speed violations.

NOTES

This is what a correct Task 5 looks like, with --battery 1:24. Left: at beat 24 the second troupe, which belongs to the captain that just died, is back at its home rows (white dots): a new captain was started and its followers wait at home. Middle: one beat later the new captain’s plan starts and the troupe dances with the others again. Right: at the end the whole picture is complete. The center audits every beat: a jump back to home is allowed exactly once after a restart, but collisions are never allowed.

GETTING STARTED

Build, test, and the order to work in

```
cd labs/swarm-phase1-skeleton
make # builds, TODOs are empty
make test # PASS/FAIL per check
make tsan # no WARNING allowed
make bench # one lock vs row locks
./center --tempo 0 --ascii 8
```

The full show needs Tasks 1 to 3 together. Until then test each piece alone: **test_queue**, **swapstress**, **gridbench**, or type protocol lines into **./leader --id 0**.

Suggested order

1. queue.c
2. grid claim, release, owner
3. center: serve_request
4. leader: thinker
5. leader: followers
6. captain restart
7. the deadlock fix

NOTES

Everything is in the skeleton repository; its README.md explains the layout. make builds center, leader and the tools, and make test runs a script that prints PASS or FAIL for each check. At the start nearly everything fails: that is expected. The whole show only runs when the queue, the planner pipeline and the followers all exist, so work in the suggested order and test the pieces separately: the queue has its own unit test, the grid has swapstress and gridbench, and you can type protocol lines into a leader by hand. If your Lab 6 kvpool.c and arena.c work, copy them into src/ and the Makefile uses them instead of the stand-in.

HAND-IN AND GRADING

What to submit, what costs you points

Submit

- Your repository (GitHub Classroom) and the link on Google Classroom
- **make test** and **make tsan** pass; Valgrind is clean
- At least **two tests of your own** in tests/run_tests.sh
- **REPORT.md**: architecture sketch, what each mutex protects, lock ordering, the deadlock write-up, captain-down handling, the bench table, AI assistance

Zero tolerance

- Busy-waiting
- An **if** around pthread_cond_wait
- An unprotected Lab 6 pool
- Zombie processes after a captain dies
- A hang on shutdown
- A missing report

Bonus up to +10%: priority queue, reader-writer lock for the grid, battery counter with a charging-station semaphore.

NOTES

Submit your repository through GitHub Classroom and put the link on Google Classroom. make test and make tsan must pass and Valgrind must be clean. Add at least two tests of your own to the test script, for example more leaders or killing two captains. The report is short but required, and it must explain, not only describe. AI use is allowed, but it must be disclosed in the report and you must be able to explain every line. The zero-tolerance list is the set of things that make a concurrent program wrong even when it appears to work. The bonus items are listed at the end of the handout.

WHEN IT HANGS

Four traps, and how to see them

A barrier nobody reaches

The barrier is shared by **all** followers. One early return or continue between the two waits, and everyone else waits forever.

A lock never released

Every unlock must happen on every path. Early returns are the usual leak, in the queue and in serve_request.

The wrong lock order

In the center: wlock first, then st_mu. The same rule fixes grid_swap: always the lower row first.

Hiding a race with sleep

Never. Run make tsan instead, and read the two stacks it prints.

A hang prints **center: TIMEOUT** after --timeout seconds. Attach with `gdb -p PID`, then `thread apply all bt`.

BIL 322 · Swarm Phase 1 lab

NOTES

Four things that will go wrong this week. One: the barrier is shared by all followers of a leader; if any of them leaves the loop between the two waits, the others wait forever. Two: a lock not released on some path, for example an early return; read your code for every return. Three: lock order. The center’s state lock and the per-leader write lock have a fixed order, commented in center.c; breaking it can deadlock. The same idea fixes the grid_swap deadlock. Four: sleeping to make a race disappear never fixes it; use ThreadSanitizer, which prints the two conflicting accesses. When the show hangs, the watchdog prints a timeout; attach gdb to the process and print all thread backtraces to see who waits for what. That output is also the evidence for the Task 4 write-up. Start with the queue.